

MCQs – SEARCHING (Computer Science, Class XII)

INTRODUCTION TO SEARCHING

1. Searching in computer science refers to
 - A) Sorting elements in a list
 - B) Locating a particular element in a collection
 - C) Deleting elements from a list
 - D) Rearranging elements in memory
 2. The result of a search operation determines
 - A) The size of the list
 - B) The type of data stored
 - C) Whether an element is present or not
 - D) The sorting order of the list
 3. If a searched element is found, searching can also determine
 - A) The frequency of the element
 - B) The memory size used
 - C) The position of the element
 - D) The data type of the element
 4. Searching is considered an important technique in computer science because
 - A) It reduces memory usage
 - B) It helps in designing algorithms
 - C) It avoids data duplication
 - D) It encrypts data
 5. A computer stores data mainly to
 - A) Display graphics
 - B) Retrieve it later when demanded
 - C) Compress files
 - D) Format storage devices
-

LINEAR SEARCH – CONCEPT

6. Linear search is also known as
 - A) Binary search
 - B) Random search
 - C) Sequential search
 - D) Indexed search
7. Linear search compares elements
 - A) Randomly
 - B) From last to first
 - C) One by one
 - D) Only at fixed positions

8. In linear search, comparison starts from
 - A) Middle element
 - B) Last element
 - C) First element
 - D) Any element
 9. Linear search is an example of
 - A) Divide and conquer technique
 - B) Exhaustive searching technique
 - C) Hashing technique
 - D) Recursive technique
 10. Linear search is most useful when the list is
 - A) Large and sorted
 - B) Small and unordered
 - C) Large and unordered
 - D) Small and sorted
 11. If no element matches the key in linear search, the search is declared
 - A) Partially successful
 - B) Deferred
 - C) Unsuccessful
 - D) Repeated
-

LINEAR SEARCH – ALGORITHM & WORKING

12. In Algorithm 6.1, the variable index is initially set to
 - A) 1
 - B) n
 - C) 0
 - D) -1
13. The loop in linear search continues while
 - A) $\text{index} > n$
 - B) $\text{index} == n$
 - C) $\text{index} < n$
 - D) $\text{index} \leq n-1$ only
14. In linear search, the comparison performed is
 - A) $\text{numList}[\text{index}] > \text{key}$
 - B) $\text{numList}[\text{index}] < \text{key}$
 - C) $\text{numList}[\text{index}] = \text{key}$
 - D) $\text{numList}[\text{index}] \neq \text{key}$
15. When a key is found, the algorithm prints
 - A) The index value
 - B) The element value
 - C) The position as $\text{index} + 1$
 - D) The number of comparisons

16. If the key is not found after traversing the list, the algorithm prints
- A) Element absent
 - B) Search terminated
 - C) Search unsuccessful
 - D) Key invalid
-

LINEAR SEARCH – EXAMPLES & OBSERVATIONS

17. If the key is the first element of the list, linear search requires
- A) n comparisons
 - B) n-1 comparisons
 - C) 1 comparison
 - D) 2 comparisons
18. If the key is the last element in the list, linear search requires
- A) n comparisons
 - B) n-1 comparisons
 - C) 1 comparison
 - D) n/2 comparisons
19. When the key is not present in the list, linear search performs
- A) One comparison
 - B) n/2 comparisons
 - C) n comparisons
 - D) Zero comparisons
20. The maximum work done by linear search occurs when
- A) Key is the first element
 - B) Key is the middle element
 - C) Key is the last element
 - D) Key is not present
21. The minimum work done by linear search occurs when
- A) Key is absent
 - B) Key is the last element
 - C) Key is the first element
 - D) List is empty
-

LINEAR SEARCH – PROGRAM BASED

22. The linear search function returns
- A) Key value
 - B) Index position
 - C) Index + 1 or None
 - D) Boolean value
23. If the function returns None, it means
- A) The list is empty
 - B) The key is not present

- C) The key is present multiple times
 - D) The list is sorted
24. In the program, user input is taken to
- A) Sort the list
 - B) Create the list
 - C) Delete elements
 - D) Reverse the list
25. The list used in the program is initially
- A) Predefined
 - B) Sorted
 - C) Empty
 - D) Random
-

BINARY SEARCH – CONCEPT

26. Binary search works efficiently only when the list is
- A) Unordered
 - B) Random
 - C) Sorted
 - D) Reversed
27. Binary search makes use of
- A) Exhaustive checking
 - B) Hash function
 - C) Ordering of elements
 - D) Stack memory
28. Binary search compares the key with
- A) First element
 - B) Last element
 - C) Middle element
 - D) Random element
29. Binary search reduces the search space by
- A) One element
 - B) Two elements
 - C) Half
 - D) One-third
30. Binary search is inspired by searching words in
- A) Newspaper
 - B) Telephone directory
 - C) Dictionary
 - D) Encyclopedia
-

BINARY SEARCH – POSSIBLE CASES

31. If the middle element matches the key, the search is
- A) Repeated
 - B) Terminated successfully
 - C) Continued
 - D) Restarted
32. If the middle element is greater than the key, the search continues in
- A) Entire list
 - B) Second half
 - C) First half
 - D) Random half
33. If the middle element is smaller than the key, the search continues in
- A) First half
 - B) Second half
 - C) Entire list
 - D) Middle portion
-

BINARY SEARCH – MID CALCULATION

34. The middle position is calculated using
- A) Normal division
 - B) Modulo operator
 - C) Floor division
 - D) Ceiling function
35. If a list has 10 elements, the middle index is
- A) 4
 - B) 5
 - C) 6
 - D) 10
36. The first element of a list has index value
- A) -1
 - B) 0
 - C) 1
 - D) Depends on list size
-

BINARY SEARCH – ITERATIONS & PERFORMANCE

37. Binary search uses the term iteration because
- A) It uses recursion
 - B) It avoids comparison
 - C) Search area is redefined each time
 - D) It sorts the list repeatedly
38. Each unsuccessful comparison in binary search
- A) Increases search space
 - B) Leaves search space unchanged

- C) Reduces search space by half
 - D) Doubles comparisons
39. Binary search requires minimum work when the key is
- A) First element
 - B) Last element
 - C) Middle element
 - D) Absent
40. Binary search requires maximum work when
- A) Key is the middle element
 - B) Key is the first element
 - C) Key is absent or near extremes
 - D) List is empty
-

BINARY SEARCH – ALGORITHM & PROGRAM

41. In binary search algorithm, initial values are
- A) first = 1, last = n
 - B) first = 0, last = n-1
 - C) first = mid
 - D) last = 0
42. The binary search loop runs while
- A) first < last
 - B) first == last
 - C) first <= last
 - D) mid <= last
43. If key is greater than middle element,
- A) last is decreased
 - B) first is increased
 - C) mid is unchanged
 - D) search stops
44. If key is smaller than middle element,
- A) first = mid + 1
 - B) last = mid - 1
 - C) mid = 0
 - D) search stops
45. The binary search function returns -1 when
- A) Key is found
 - B) List is empty
 - C) Key is not found
 - D) List is unsorted
-

APPLICATIONS OF BINARY SEARCH

46. Binary search can be used for
- A) Searching a dictionary
 - B) Telephone directory
 - C) Finding minimum/maximum in sorted list
 - D) All of the above
47. Modified binary search techniques are used in
- A) Image editing
 - B) Database indexing
 - C) File compression only
 - D) Email services
-

SEARCH BY HASHING – CONCEPT

48. Hashing allows searching in
- A) Linear time
 - B) Logarithmic time
 - C) One step
 - D) Multiple passes
49. Hashing uses a
- A) Sorting algorithm
 - B) Hash function
 - C) Binary tree
 - D) Stack
50. A hash function generates
- A) Data type
 - B) Index value
 - C) Sorted list
 - D) Key
51. The new list created using hashing is called
- A) Search list
 - B) Sorted list
 - C) Hash table
 - D) Linked list
-

HASH TABLE & HASH FUNCTION

52. Each index of a hash table can hold
- A) Multiple items
 - B) Only one item
 - C) No items
 - D) Two items
53. Hash table indices start from
- A) 1
 - B) -1

- C) 0
 - D) Depends on size
54. The size of hash table can be
- A) Equal to list size only
 - B) Smaller than list size only
 - C) Larger than list size
 - D) Always fixed
55. The remainder method hash function is
- A) $h(\text{key}) = \text{key} // \text{size}$
 - B) $h(\text{key}) = \text{key} * \text{size}$
 - C) $h(\text{key}) = \text{key} \% \text{size}$
 - D) $h(\text{key}) = \text{size} \% \text{key}$
-

HASHING – SEARCH & PERFORMANCE

56. Hash-based searching requires
- A) Multiple comparisons
 - B) n comparisons
 - C) One comparison
 - D) $\log n$ comparisons
57. Hashing performance is independent of
- A) Hash function
 - B) List size
 - C) Table size
 - D) Key value
-

COLLISION & PERFECT HASHING

58. Collision occurs when
- A) Key is not found
 - B) Two elements map to same index
 - C) Hash table is full
 - D) Key is repeated
59. Collision resolution refers to
- A) Deleting keys
 - B) Rehashing table
 - C) Placing items with same hash value
 - D) Sorting hash table
60. A perfect hash function ensures
- A) Fast sorting
 - B) Zero collisions
 - C) Multiple keys per index
 - D) Larger hash table

SUMMARY & CONCEPTUAL MCQs

61. Linear search checks elements
- A) Randomly
 - B) One at a time
 - C) By dividing list
 - D) Using hash function
62. Binary search is faster than linear search because
- A) It skips comparisons
 - B) It halves the search space
 - C) It uses recursion
 - D) It uses hashing
63. Hashing is efficient because
- A) It sorts data
 - B) It reduces memory
 - C) Index computation time is constant
 - D) It avoids collisions
64. When two elements map to the same slot, it is called
- A) Iteration
 - B) Comparison
 - C) Collision
 - D) Overflow
65. Collision resolution techniques are
- A) Discussed in detail
 - B) Not required
 - C) Beyond scope of the book
 - D) Mandatory for binary search